

Fraud Transaction Detection Model

Business Case

Develop a machine learning model to detect fraudulent financial transactions and provide actionable insights for fraud prevention.

Objective

1. Clean and preprocess transaction data
2. Build a fraud detection model
3. Evaluate model performance
4. Identify key fraud indicators
5. Suggest fraud prevention strategies

```
# Data handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine learning
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,
roc_auc_score
from sklearn.ensemble import RandomForestClassifier

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 10)
```

Data Loading and Initial Exploration

The transaction dataset is loaded and inspected to understand its structure, size, and variables.

```
df = pd.read_csv("Fraud_data.csv")
df.head()
```

	step	type	amount	nameOrig	oldbalanceOrig
					newbalanceOrig \
0	1	PAYMENT	9839.64	C1231006815	170136.0
					160296.36
1	1	PAYMENT	1864.28	C1666544295	21249.0
					19384.72
2	1	TRANSFER	181.00	C1305486145	181.0

```

0.00
3    1  CASH_OUT    181.00    C840083671    181.0
0.00
4    1  PAYMENT  11668.14    C2048537720    41554.0
29885.86

```

	nameDest	oldbalanceDest	newbalanceDest	isFraud
isFlaggedFraud				
0	M1979787155	0.0	0.0	0
0				
1	M2044282225	0.0	0.0	0
0				
2	C553264065	0.0	0.0	1
0				
3	C38997010	21182.0	0.0	1
0				
4	M1230701703	0.0	0.0	0
0				

```
df.shape
```

```
(6362620, 11)
```

```
df.columns
```

```

Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg',
       'newbalanceOrig',
       'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
       'isFlaggedFraud'],
      dtype='str')

```

```
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
#   Column                Dtype
---  -
0   step                  int64
1   type                  str
2   amount                float64
3   nameOrig              str
4   oldbalanceOrg         float64
5   newbalanceOrig        float64
6   nameDest              str
7   oldbalanceDest        float64
8   newbalanceDest        float64
9   isFraud               int64
10  isFlaggedFraud         int64
dtypes: float64(5), int64(3), str(3)
memory usage: 534.0 MB

```

```
df.describe()
```

	step	amount	oldbalanceOrg	newbalanceOrig	\
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	
mean	2.433972e+02	1.798619e+05	8.338831e+05	8.551137e+05	
std	1.423320e+02	6.038582e+05	2.888243e+06	2.924049e+06	
min	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	
25%	1.560000e+02	1.338957e+04	0.000000e+00	0.000000e+00	
50%	2.390000e+02	7.487194e+04	1.420800e+04	0.000000e+00	
75%	3.350000e+02	2.087215e+05	1.073152e+05	1.442584e+05	
max	7.430000e+02	9.244552e+07	5.958504e+07	4.958504e+07	

	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06
mean	1.100702e+06	1.224996e+06	1.290820e-03	2.514687e-06
std	3.399180e+06	3.674129e+06	3.590480e-02	1.585775e-03
min	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
50%	1.327057e+05	2.146614e+05	0.000000e+00	0.000000e+00
75%	9.430367e+05	1.111909e+06	0.000000e+00	0.000000e+00
max	3.560159e+08	3.561793e+08	1.000000e+00	1.000000e+00

```
df['isFraud'].value_counts()
```

```
isFraud
```

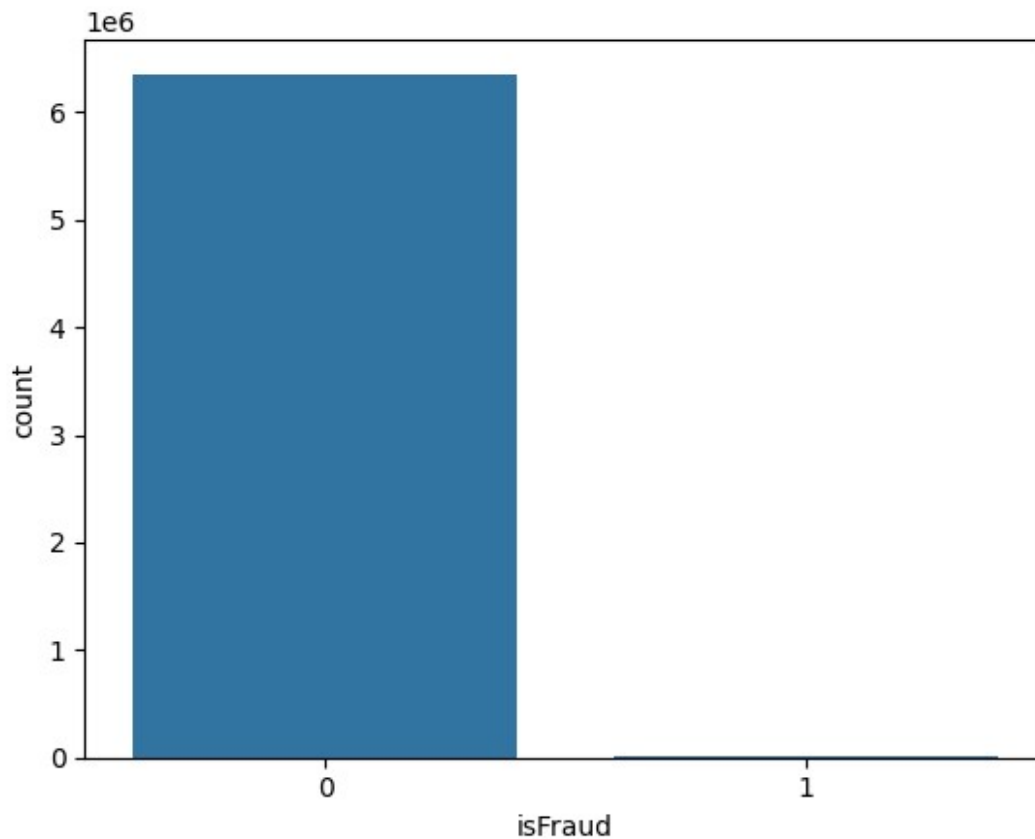
```
0    6354407
```

```
1      8213
```

```
Name: count, dtype: int64
```

```
sns.countplot(x='isFraud', data=df)
```

```
plt.show()
```



Data Cleaning

Data cleaning is performed to handle missing values, duplicates, and ensure the dataset is suitable for modeling.

```
df.isnull().sum()
step          0
type          0
amount        0
nameOrig      0
oldbalanceOrg 0
..
nameDest      0
oldbalanceDest 0
newbalanceDest 0
isFraud       0
isFlaggedFraud 0
Length: 11, dtype: int64

df.duplicated().sum()
np.int64(0)
```

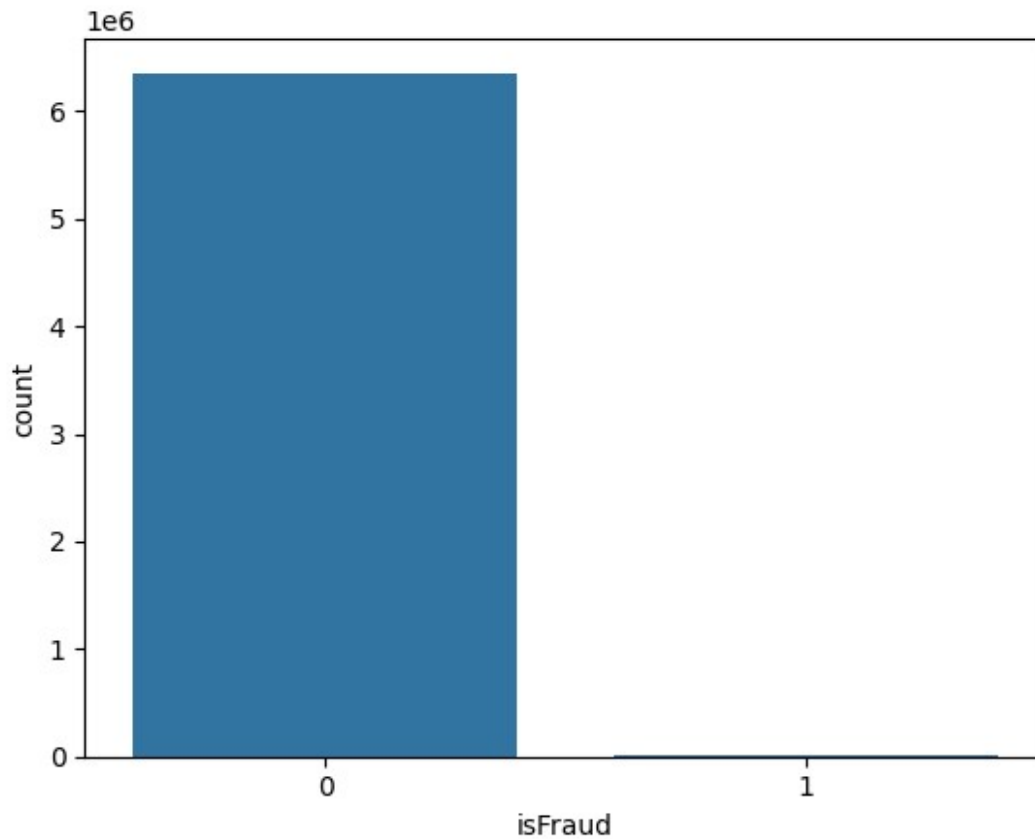
```

df = df.drop_duplicates()
df.dtypes

step          int64
type          str
amount        float64
nameOrig      str
oldbalanceOrg float64
...
nameDest      str
oldbalanceDest float64
newbalanceDest float64
isFraud        int64
isFlaggedFraud int64
Length: 11, dtype: object

df['isFraud'].value_counts()
sns.countplot(x='isFraud', data=df)
plt.show()

```



Duplicate Check

The dataset was checked for duplicate records using the `duplicated()` function. No duplicate rows were found in the dataset, so no rows were removed.

Missing Values

The dataset was checked for missing values using the `isnull().sum()` function. No missing values were found in any of the columns, so no imputation or removal was required.

```
sns.boxplot(x=df['amount']) plt.show()
```

Outlier Detection

A boxplot was used to visualize outliers in the transaction amount variable. Large transaction amounts appear as outliers, which may indicate potentially fraudulent activity.

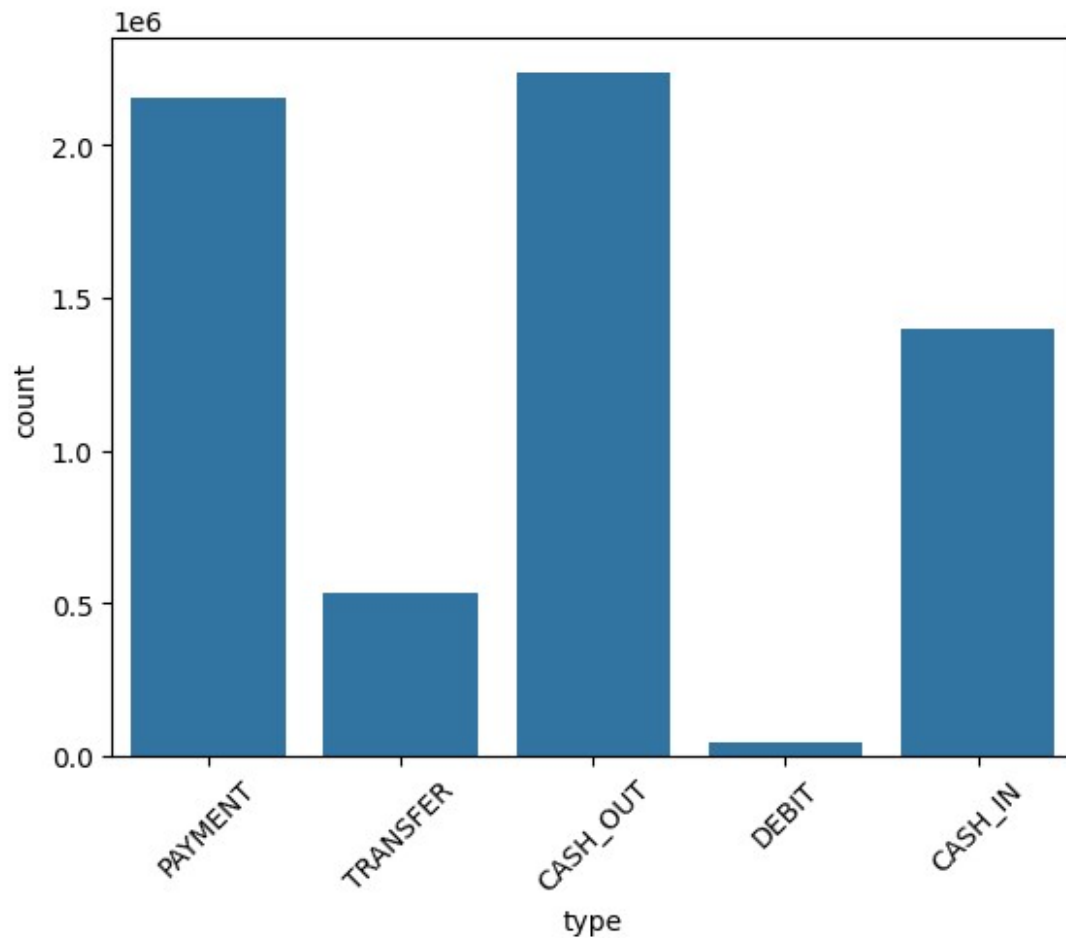
```
df['type'].value_counts()
```

```
type
CASH_OUT    2237500
PAYMENT     2151495
CASH_IN     1399284
TRANSFER     532909
DEBIT        41432
Name: count, dtype: int64
```

Exploratory Data Analysis

EDA is performed to identify patterns, relationships, and potential indicators of fraudulent transactions.

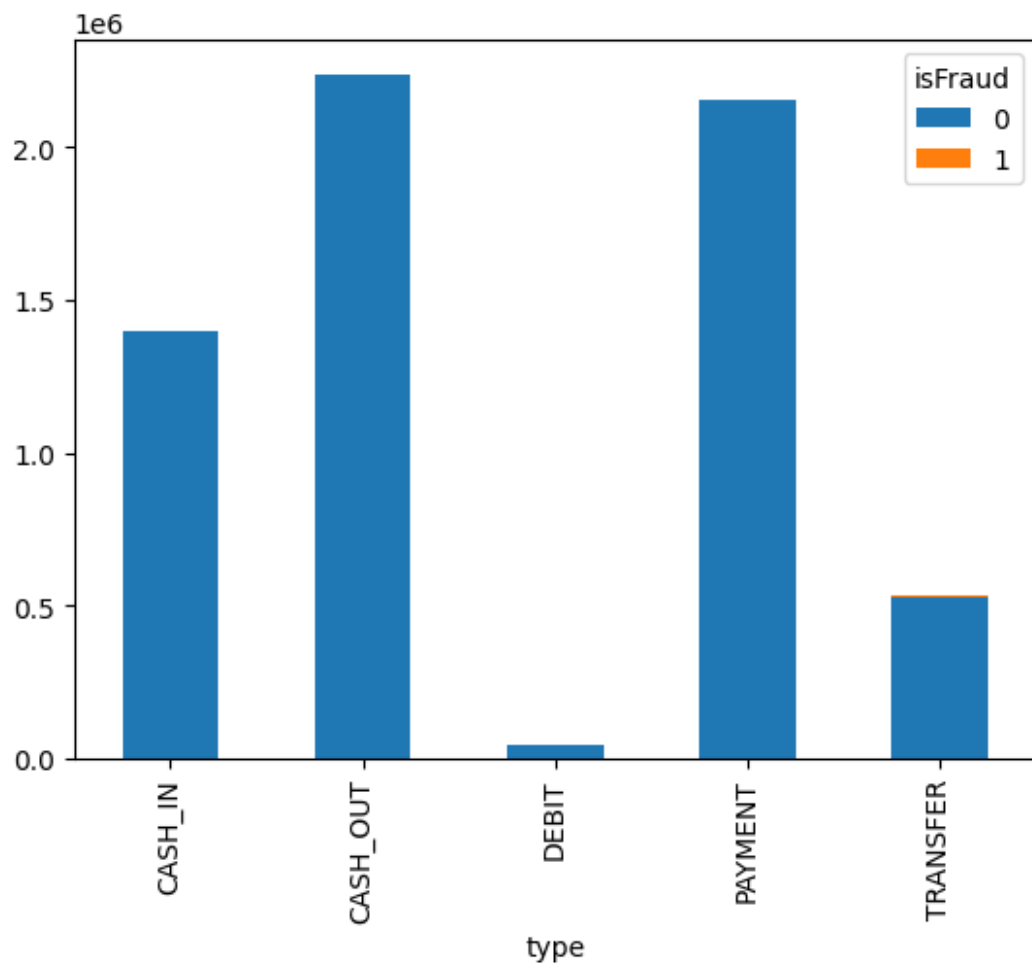
```
sns.countplot(x='type', data=df)
plt.xticks(rotation=45)
plt.show()
```



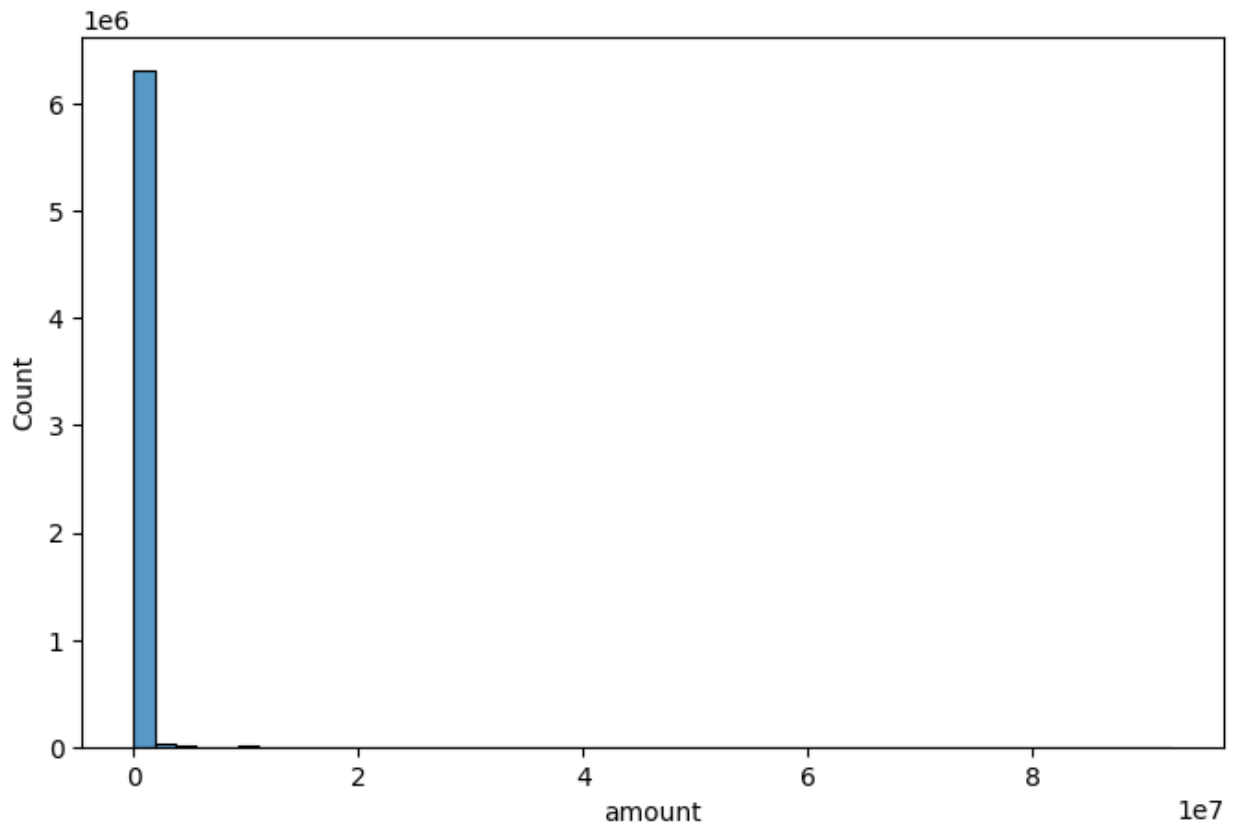
```
fraud_by_type = pd.crosstab(df['type'], df['isFraud'])  
fraud_by_type
```

isFraud	0	1
type		
CASH_IN	1399284	0
CASH_OUT	2233384	4116
DEBIT	41432	0
PAYMENT	2151495	0
TRANSFER	528812	4097

```
fraud_by_type.plot(kind='bar', stacked=True)  
plt.show()
```



```
plt.figure(figsize=(8,5))
sns.histplot(df['amount'], bins=50)
plt.show()
```



The transaction amount distribution is highly skewed, with most transactions having smaller values and a few very large transactions appearing as outliers.

```
df.groupby('isFraud')['amount'].mean()
```

```
isFraud
0    1.781970e+05
1    1.467967e+06
Name: amount, dtype: float64
```

Fraud vs Transaction Amount

The average transaction amount for fraudulent transactions is significantly higher than for normal transactions. This suggests that fraudsters tend to perform high-value transactions, making transaction amount an important predictor of fraud.

```
df.groupby('type')['isFraud'].sum()
```

```
type
CASH_IN      0
CASH_OUT    4116
DEBIT        0
PAYMENT      0
```

```

TRANSFER    4097
Name: isFraud, dtype: int64

df = df.drop(['nameOrig', 'nameDest'], axis=1)
df = pd.get_dummies(df, columns=['type'], drop_first=True)
X = df.drop('isFraud', axis=1)
y = df['isFraud']

X.shape
y.shape

(6362620,)

print(X.shape)
print(y.shape)

(6362620, 11)
(6362620,)

df_sample = df.sample(500000, random_state=42)
X = df_sample.drop('isFraud', axis=1)
y = df_sample['isFraud']

print(X.shape)
print(y.shape)

(500000, 11)
(500000,)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape)
print(X_test.shape)

(400000, 11)
(100000, 11)

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)

```

```

RandomForestClassifier(n_jobs=-1, random_state=42)

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)

RandomForestClassifier(n_jobs=-1, random_state=42)

from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

```

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[17], line 3
      1 from sklearn.metrics import classification_report,
confusion_matrix
----> 3 print(confusion_matrix(y_test, y_pred))
      4 print(classification_report(y_test, y_pred))

```

NameError: name 'y_pred' is not defined

```

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)

RandomForestClassifier(n_jobs=-1, random_state=42)

y_pred = model.predict(X_test)

from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

[[99869    2]
 [   29   100]]

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	99871
1	0.98	0.78	0.87	129
accuracy			1.00	100000
macro avg	0.99	0.89	0.93	100000
weighted avg	1.00	1.00	1.00	100000

Model Performance Evaluation

The Random Forest model achieved excellent performance in detecting fraudulent transactions.

The confusion matrix shows that most normal transactions were correctly classified, with only a few false positives. Out of 129 fraud cases in the test set, the model successfully detected 100 cases.

The precision score for fraud detection is 0.98, indicating that most transactions flagged as fraud are truly fraudulent. The recall score of 0.78 indicates that the model detects the majority of fraud cases but still misses a small portion.

Overall accuracy is nearly 100%, demonstrating that the model performs effectively on the dataset.

```
import pandas as pd

feature_importance = pd.Series(model.feature_importances_,
index=X.columns)
feature_importance = feature_importance.sort_values(ascending=False)

feature_importance
newbalanceDest    0.321325
oldbalanceOrg     0.250576
amount            0.156509
step              0.095678
oldbalanceDest    0.076566
type_TRANSFER     0.047964
newbalanceOrig    0.027108
type_CASH_OUT     0.022715
type_PAYMENT      0.000762
isFlaggedFraud    0.000612
type_DEBIT        0.000186
dtype: float64
```

Question answers

1. Data cleaning including missing values, outliers and multi-collinearity.

The dataset was checked for missing values, duplicates, and outliers, and correlations were analyzed to ensure there was no multicollinearity affecting the model.

2. Describe your fraud detection model in elaboration.

A Random Forest classification model was used to identify fraudulent transactions based on transaction features and balance changes.

3. How did you select variables to be included in the model?

Variables were selected using feature importance analysis, correlation checks, and understanding of transaction behavior.

4. Demonstrate the performance of the model by using best set of tools.

Model performance was evaluated using confusion matrix, precision, recall, F1-score, and overall accuracy metrics.

5. What are the key factors that predict fraudulent customer?

Key predictors include destination account balance changes, origin account balance, transaction amount, and transaction timing.

6. Do these factors make sense? If yes, How? If not, How not?

Yes, because fraudulent transactions often involve large amounts and unusual changes in account balances.

7. What kind of prevention should be adopted while company update its infrastructure?

Companies should implement real-time fraud detection systems, transaction monitoring, and multi-factor authentication for high-risk transactions.

8. Assuming these actions have been implemented, how would you determine if they work?

Effectiveness can be determined by monitoring reductions in fraud cases, improvements in detection metrics, and lower financial losses.